

PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS

Thaís dos Reis Soares
Matrícula: 825322

Análise de Eficácia de Testes com Teste de Mutação

Disciplina: Teste de Software

1. Introdução

Este relatório apresenta uma análise da eficácia da suíte de testes da biblioteca `operacoes.js`, utilizando a técnica de teste de mutação com a ferramenta StrykerJS. O objetivo é investigar a discrepância entre a métrica de cobertura de código e a real qualidade dos testes, demonstrando como 100% de cobertura nem sempre garante que um sistema está protegido contra erros. Além disso, também serão apresentadas as melhorias implementadas nos testes de unidade para elevar a pontuação de mutação e a relevância dessa metodologia no desenvolvimento de software.

2. Análise Inicial

Após a execução inicial dos testes, a cobertura de código obtida foi igual a 100% para funções, o que significa que todas foram executadas, 98,64% para as linhas e 85,41% para os comandos, conforme apresentado na figura 1.

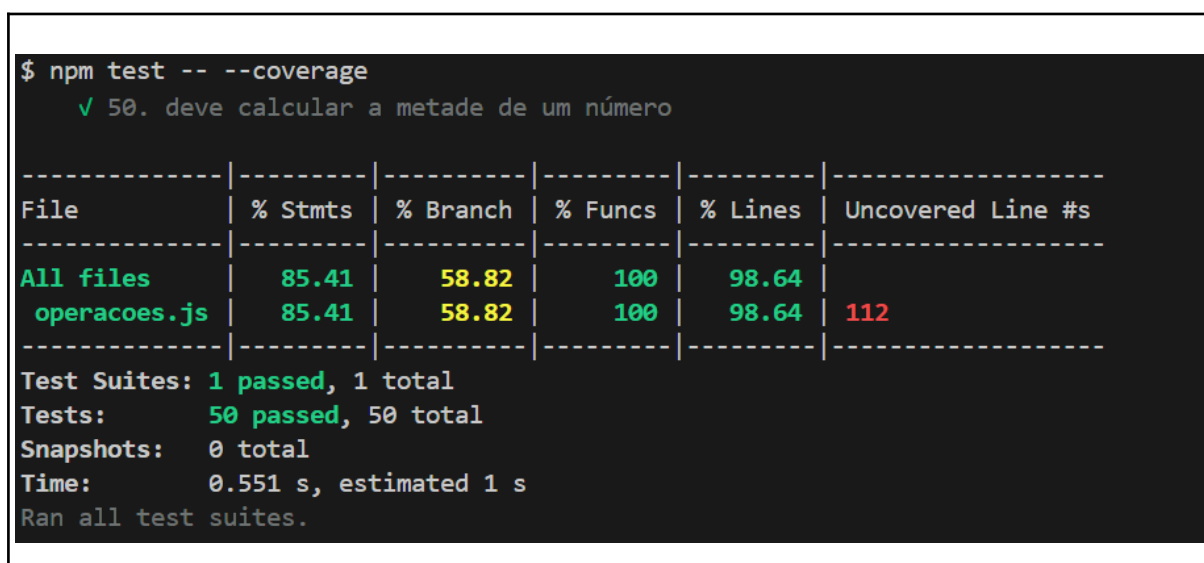


Figura 1

Já a pontuação de mutação inicial foi igual a 73,71%, com 44 mutantes sobreviventes e 12 mutantes classificados como *no coverage*, como ilustra a figura 2.

157										44		12	
File / Directory	Mutation Score		Killed	Survived	Timeout	No coverage	Ignored	Runtime errors	Compile errors	Detected	Undetected	Total	
	Of total	Of covered											
All files	73.71	78.11	154	44	3	12	0	0	0	157	56	213	
js operacoes.js	73.71	78.11	154	44	3	12	0	0	0	157	56	213	

Figura 2

Esses resultados revelam que a qualidade de uma suíte de testes não pode ser medida apenas pela cobertura de código, pois apesar dela ter sido alta, muitos mutantes sobreviveram. Isso evidencia que, apesar da alta cobertura, vários cenários e comportamentos não estavam sendo testados, o que poderia deixar passar erros no software.

3. Análise de Mutantes Críticos

A seguir, serão apresentados três mutantes críticos sobreviventes da primeira execução.

3.1 - Mutante na função raizQuadrada

```
-   if (n < 0) throw new Error('Não é possível calcular a raiz quadrada de um número negativo.');
```

```
+   if (false) throw new Error('Não é possível calcular a raiz quadrada de um número negativo.');
```

Figura 3

Neste caso, a mutação trocou a condição do if de “n < 0” para o valor booleano false. O mutante sobreviveu porque a suíte de testes considerava apenas a raiz quadrada de números positivos, não incluindo casos com valores negativos. Dessa forma, não houve validação do comportamento esperado de lançamento de exceção para entradas inválidas.

3.2 - Mutante na função isPar

```
- function isPar(n) { return n % 2 === 0; }  
+ function isPar(n) { return true; }
```

Figura 4

Esta mutação alterou o retorno da função, tirando a condição que verificava se um número é par para o valor booleano true. Ela sobreviveu aos testes porque a suíte contemplava apenas casos com números pares, não incluindo testes com números ímpares. Assim, o comportamento incorreto não foi detectado.

3.3 - Mutante na função celsiusParaFahrenheit

```
- function celsiusParaFahrenheit(celsius) { return (celsius * 9/5) + 32; }  
+ function celsiusParaFahrenheit(celsius) { return (celsius * 9 * 5) + 32; }
```

Figura 5

Esta mutação alterou a forma como a conversão de Celsius para Fahrenheit é calculada. Ela sobreviveu porque a suíte de testes incluía apenas o caso em que Celsius era igual a 0, situação em que a multiplicação não impacta o resultado final. Assim, tanto o código original quanto o mutante produziam a mesma saída nesse cenário específico, permitindo que o erro não fosse detectado.

4. Solução Implementada

Para matar o mutante na função raizQuadrada, foi necessário incluir um teste com valores negativos, definindo o lançamento de uma exceção como comportamento esperado. Como o código mutante não realizava esse tratamento corretamente, o teste falhou, permitindo assim a detecção e eliminação do mutante, como mostra a figura 6.

```
expect(() => raizQuadrada(-4)).toThrow('Não é possível calcular a raiz quadrada de um número negativo.');
```

Figura 6

Já para o mutante na função `isPar`, foi necessário adicionar um teste com um número ímpar, que esperasse como retorno o valor booleano `false`, e como isso não acontecia no código mutante, foi possível eliminá-lo.

```
expect(isPar(101)).toBe(false);
```

Figura 7

Por fim, para o mutante da função `celsiusParaFahrenheit`, foi necessário um teste com um valor para `celsius` diferente de zero. Isso garantiu que o resultado esperado fosse distinto daquele produzido pelo código mutante, permitindo identificar a falha e, conseqüentemente, matar o mutante.

```
expect(celsiusParaFahrenheit(100)).toBe(212);
```

Figura 8

5. Resultados Finais

Ao final das melhorias realizadas na suíte de testes, a pontuação de mutação era de 98,12%, apresentando uma melhoria considerável em comparação ao estado inicial.

209												4
File / Directory	Mutation Score		Killed	Survived	Timeout	No coverage	Ignored	Runtime errors	Compile errors	Detected	Undetected	Total
	Of total	Of covered										
All files	98.12	98.12	206	4	3	0	0	0	0	209	4	213
js operacoes.js	98.12	98.12	206	4	3	0	0	0	0	209	4	213

Figura 9

6. Conclusão

Com base no que foi apresentado neste relatório, é possível observar a contribuição significativa do teste de mutação para a melhoria da qualidade da suíte de testes. Inicialmente, embora a cobertura de código fosse alta, a eficácia dos testes era baixa, pois diversos cenários relevantes não estavam sendo contemplados, o que

poderia permitir falhas no sistema. Isso ficou evidenciado pela baixa pontuação de mutação no início da análise.

Com a adoção do teste de mutação, foi possível identificar lacunas na suíte de testes e incluir novos casos que validam diferentes comportamentos da aplicação. Como resultado, a pontuação de mutação aumentou de 73,71% para 98,12%, demonstrando uma melhora significativa na capacidade dos testes de detectar falhas. Dessa forma, a suíte de testes tornou-se consideravelmente mais robusta e eficaz.